

JAVA SWING LAYOUT-MANAGER ENTSCHEIDUNGSFINDER

Visueller Leitfaden & Code-Templates für die 60-Minuten-Klausur

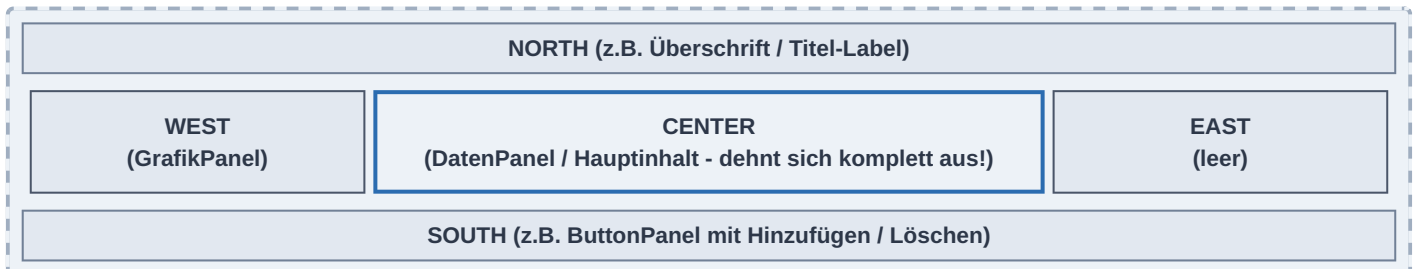
KLAUSUR-GEHEIMNISS FÜR SCHNELLES LAYOUT: Van der Kamp nutzt fast immer eine Kombination! Das Hauptfenster (JFrame) nutzt **BorderLayout**, um die Sub-Panels (oben/mitte/unten) zu platzieren. Die einzelnen Sub-Panels nutzen dann entweder **FlowLayout** (für einfache Zeilen) oder **GridLayout** (für Formulare mit Labels und Textfeldern).

1. Die Layout-Entscheidungsmatrix

Layout	Wann benutzen? (Typisches Klausurszenario)	Hauptmerkmal / Verhalten
BorderLayout	Immer für das Hauptfenster (JFrame). Wenn die Skizze ein linkes Grafikpanel, ein mittleres Datenpanel und ein unteres Buttonpanel zeigt.	Teilt den Raum in 5 Zonen (NORTH, SOUTH, EAST, WEST, CENTER). Center dehnt sich automatisch in den restlichen Raum aus.
FlowLayout	Für das ButtonPanel unten oder einfache, zentrierte/linksbündige Zeilen (z.B. Eingabezeile mit einem Label + ein Feld).	Ordnet Komponenten nacheinander in einer Zeile an. Wenn kein Platz mehr ist, erfolgt ein automatischer Umbruch. Komponenten behalten Wunschgröße.
GridLayout	Für strukturierte Eingabemasken (Formulare) . Wenn exakt untereinander ausgerichtete Zeilen von Label und Textfeld gefordert sind.	Erzwingt ein Raster aus Zeilen und Spalten. Alle Zellen sind exakt gleich groß! Komponenten werden gestreckt.

2. Visualisierung & Code-Schablone: BorderLayout

Teilt den Container in feste Himmelsrichtungen auf. Nicht besetzte Richtungen kollabieren auf 0 Pixel Breite/Höhe.

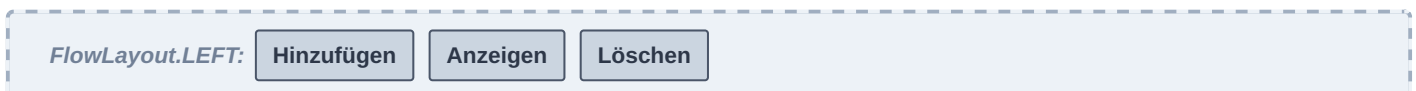


```
// Standard-Layout für JFrame. Muss im Konstruktor gesetzt werden:
this.setLayout(new BorderLayout(5, 5)); // 5px horizontaler & vertikaler Abstand

// Hinzufügen der Sub-Panels mit Richtungsangabe:
this.add(new GrafikPanel(), BorderLayout.WEST);
this.add(new AnzeigePanel(), BorderLayout.CENTER); // Nimmt den gesamten Restplatz ein
this.add(new ButtonPanel(), BorderLayout.SOUTH);
```

3. Visualisierung & Code-Schablone: FlowLayout

Komponenten behalten ihre Standardgröße (`getPreferredSize()`) und fließen von links nach rechts.



```
// Konstruktor des Sub-Panels (z.B. ButtonPanel):
public ButtonPanel() {
    // Ausrichtung linksbündig mit 10px horizontalem und 5px vertikalem Abstand
    this.setLayout(new FlowLayout(FlowLayout.LEFT, 10, 5));

    this.add(buttonHinzufuegen);
    this.add(buttonAnzeigen);
    this.add(buttonLoeschen);
}
```

4. Visualisierung & Code-Schablone: GridLayout

Erzwingt ein striktes, gleichmäßiges Gitter. Perfekt für bündige Formular-Eingaben.

Label "Name:"	[Textfeld Name (ausgefüllt)]
Label "Datum:"	[Textfeld Datum (ausgefüllt)]

```
// Konstruktor des Sub-Panels (z.B. AnzeigePanel für ein Formular):
public AnzeigePanel() {
    // Parameter: GridLayout(Anzahl_Zeilen, Anzahl_Spalten, H-Gap, V-Gap)
    // Bei 0 Zeilen wächst das Grid dynamisch basierend auf den Spalten!
    this.setLayout(new GridLayout(2, 2, 10, 10));

    // WICHTIG: Die Reihenfolge des Hinzufügens bestimmt die Position (Zeile für Zeile)!
    this.add(new JLabel("Name:"));           // Zeile 1, Spalte 1
    this.add(this.textFieldName);           // Zeile 1, Spalte 2

    this.add(new JLabel("Datum:"));         // Zeile 2, Spalte 1
    this.add(this.textFieldDatum);         // Zeile 2, Spalte 2
}
```

ACHTUNG BEI GRIDLAYOUT: Da alle Zellen gradenlos auf dieselbe Größe gestreckt werden, sieht ein einzelner Button in einem GridLayout riesig und unnatürlich aus. Nutze GridLayout *nur* für gleichwertige Paarungen (wie Label + Textfeld). Für Buttons in einer Reihe nutzt du immer das FlowLayout!